

mor.money Technical Whitepaper

Verifiable Intent Execution and Off-Chain Dark Pool Settlement via Arbitrum Stylus and zkML

Author: Babak Safari (Founder, mor.money)

Date: September 2026 | **Version:** 1.0-Draft

Infrastructure: Arbitrum Stylus (Rust/WASM) • zkML Verifier • FastAPI Intent Parser

ABSTRACT

Modern decentralized finance (DeFi) lacks execution confidentiality and robust protection against Maximal Extractable Value (MEV) for large block trades. Public EVM mempools expose institutional orders to front-running, sandwich attacks, and execution slippage. We present **mor.money**, an institutional-grade execution platform combining natural language intent parsing, off-chain dark pool order matching, and zero-knowledge Machine Learning (zkML) circuit verification settling on-chain via Arbitrum Stylus (Rust/WASM).

1. Introduction

1.1 The MEV Tax on Institutional Capital

Public mempools in Ethereum-compatible blockchain ecosystems act as transparent order books where predatory searchers execute sandwich attacks and front-running strategies. For institutional desks attempting to rebalance portfolios or execute large block trades, this exposure introduces significant execution drag and performance leakage.

1.2 Limitations of Standard EVM Execution Models

Traditional Solidity smart contracts running on the Ethereum Virtual Machine (EVM) present two major bottlenecks: high gas overhead during complex computation and complete transparency of transaction parameters prior to block inclusion. Existing solutions, such as batch auctions, introduce multi-block latency constraints incompatible with dynamic algorithmic strategies.

1.3 The mor.money Paradigm

mor.money introduces a hybrid off-chain/on-chain architecture designed to deliver sub-second response times, complete privacy against MEV bots, and cryptographically verifiable execution bounds without exposing proprietary strategy logic.

2. System Architecture & Microservices

The platform operates as a containerized microservice stack managed via Docker Compose and systemd daemons, structured into four main operational layers:

Service Component	Container / Module	Role & Architecture Specs
Intent Parser Engine	<code>fastapi-intent-parser</code>	Converts natural language and API inputs into structured execution vectors.
Dark Pool Matcher	<code>dark-pool-matcher</code>	Private off-chain order book matching engine preventing mempool exposure.
zkML Circuit Verifier	<code>zkml-verifier</code>	Generates zero-knowledge proofs confirming trade bounds follow model risk rules.
Stylus On-Chain Settlement	Arbitrum Stylus (Rust)	WASM-compiled smart contracts executing on Arbitrum with minimal gas overhead.
Telemetry Frontend	<code>streamlit-dashboard</code>	Real-time multi-DEX route tracking and order flow audit interface (Port 8501).

3. Technical Workflow & Execution Pipeline

- Intent Parameterization:** A user or automated algorithm submits an execution query. The `fastapi-intent-parser` parses the constraints (e.g., maximum slippage S_{max} , deadline T , and target venue priority).
- Off-Chain Dark Pool Matching:** Orders enter the `dark-pool-matcher` where counterparty flow is matched internally off-chain. If internal counterparty liquidity is insufficient, the system falls back to external aggregated DEX venues.
- zkML Proof Generation:** Before broadcasting to the settlement layer, the `zkml-verifier` evaluates execution metrics against risk bounds, outputting a cryptographic proof π_{zk} that confirms policy compliance without exposing parameter weights.
- Arbitrum Stylus Settlement:** The transaction and proof π_{zk} settle via Arbitrum Stylus Rust contracts, benefiting from WebAssembly speed and low gas execution.

4. Formal Verification & Security Mechanics

Let O represent an incoming block trade intent defined by $O = \{A, Q, P_{limit}\}$, where A is the asset, Q is the quantity, and P_{limit} is the limit price threshold. The dark pool matcher ensures that for any matched pair (O_i, O_j) :

$$P_{execution} \in [P_{bid}, P_{ask}] \wedge MempoolExposure(O_i) = \emptyset$$

By enforcing $MempoolExposure = \emptyset$ prior to state finality, predatory MEV searchers receive zero signal regarding pending trade volume.

5. Conclusion & Deployment Roadmap

mor.money solves the fundamental trade-off between privacy, execution speed, and verifiable security in decentralized finance. By coupling off-chain matching with Arbitrum Stylus and zkML verification, institutional traders gain access to zero-MEV execution at native performance scales.